

Weekly Snippet

Author: Austin Li

Created Date: May 28th 2026

Last updated Date: July 5th 2026

May 28, 2026

Week 1:

Progress and issues resolved

Backend service

- Set up the FastAPI backend skeleton for the Web3 Proof-of-Human Trust API.
- Added the health check endpoint `/`.
- Added the wallet ingestion endpoint `/check\_wallet`.
- Fixed setup issues with uvicorn and missing module imports.
- Tested the endpoint locally with FastAPI docs and curl.

Repo setup

- Created and linked the GitHub repository.
- Added `.gitignore` to exclude local environment files.
- Added `.env.example` as a template for future API keys.
- Pushed the Week 1 backend skeleton to GitHub.

Documentation

- Added initial README instructions.
- Added example request and response for `/check\_wallet`.
- Added initial architecture notes for wallet ingestion, scoring pipeline, and future blockchain data integration.

June 5th 2026

Week 2:

Feature Extraction + Scoring using sequential rule + Storage into Database

- Built wallet feature extraction pipeline.
- Integrated live Etherscan data for balance, transaction count, contract checks, and transaction samples.

- Computed features like balance level, activity level, wallet age, activity frequency, counterparty count, and risk flags.
- Built heuristic scoring service for human likelihood, trust tier, confidence score, and score breakdown.
- Added `/extract_features`, `/score_wallet`, and compact `/check_wallet` endpoints.
- Added Supabase/Postgres models for wallets, feature snapshots, and score snapshots.
- Stored wallet, feature, and score records in Supabase.
- Added unit tests for validation, scoring, auth, and wallet flow.

June 11th 2026

Week 3:

Trust API, Proof System, API Docs, Auth, Rate Limiting, and Error Handling

Trust API + proof system

- Built the `/generate_proof` endpoint to create reusable wallet trust proofs.
- Added proof fields including proof ID, proof version, active status, revocable flag, issued time, valid-until time, and behavior fingerprint hash.
- Added `/verify_proof` so another app can check whether a generated proof is active, expired, not found, or unavailable.
- Stored generated proof snapshots in Supabase/Postgres and linked them back to the wallet ID.
- Updated proof responses to include human likelihood, trust tier, confidence score, and proof metadata.

API security + reliability

- Added API key authentication using the `X-API-Key` request header.
- Added rate limiting to protected endpoints to prevent too many requests in a short time window.
- Added centralized error handling for wallet validation errors and unexpected server errors.
- Added backend logging for request method, endpoint path, status code, and request timing.

API documentation

- Updated `docs/api.md` with authentication instructions, rate limit behavior, endpoint roles, request examples, and response examples.
- Added a Week 3 proof flow showing how to check a wallet, generate a proof, and verify a proof.
- Documented `/check_wallet`, `/extract_features`, `/score_wallet`, `/generate_proof`, and `/verify_proof`.

Testing

- Added tests for rate limiting.
- Added tests for proof verification states: active, expired, and not found.

- Updated proof tests to match the new proof schema.
- Confirmed the backend test suite passed with 24 tests.
- Verified proof generation and proof verification against stored Supabase records.

June 18th 2026

Week 4:

Dashboard, Simulation, Performance Benchmarking, and Initial Deployment

Dashboard

- Built a React-based internal dashboard for wallet reputation checks.
- Added UI actions for Check Wallet, Generate Proof, Verify Proof, and Load Dashboard.
- Connected the frontend dashboard to the FastAPI backend using API base URL and `X-API-Key`.
- Added dashboard views for total wallets, score snapshots, proof snapshots, trust tier distribution, human likelihood distribution, recent wallets, and flagged wallets.

Simulation + performance

- Added `scripts/simulate\_developer\_usage.py` to simulate an external developer calling the API workflow.
- Added `scripts/benchmark\_api.py` to measure response latency across readiness and dashboard endpoints.
- Added API client helper code for reusable scripted requests.
- Added Week 4 simulation and performance documentation.

Deployment

- Deployed the FastAPI backend to Railway.
- Deployed the React frontend/dashboard to Railway as a separate frontend service.
- Configured Railway service domains for backend and frontend.
- Fixed deployment issues around frontend root directory, Vite host allowlist, and backend start command.
- Added `/health` and `/debug/env` readiness endpoints for deployment checks.

Testing

- Added tests for dashboard service behavior.
- Added tests for Week 4 simulation and benchmark scripts.
- Confirmed the backend test suite passed with 35 tests after Week 4 readiness work.

July 5th 2026

Week 5:

Data enrichment

- Added Alchemy enrichment support through a new Alchemy client.
- Integrated Alchemy asset transfer sampling for incoming and outgoing wallet activity.
- Expanded Etherscan usage to include ERC721 and ERC1155 NFT transfer samples.
- Continued using Etherscan for native balance, transaction count, contract code checks, and normal transaction samples.
- Added optional enrichment behavior so the pipeline still works if Alchemy is unavailable.

Advanced feature extraction

- Added transaction diversity ratio.
- Added contract interaction count and contract interaction ratio.
- Added normalized transaction entropy to measure repeated transaction behavior.
- Added max transactions per hour and max transactions per day for burst detection.
- Added repeated contract loop count.
- Added NFT activity features including ``nft_transfer_sample_size`` and ``has_nft_activity``.
- Added Alchemy-specific feature fields such as transfer sample size, unique counterparty count, and transfer categories.

Bot detection rules + scoring

- Added burst activity heuristics.
- Added repeated contract loop heuristics.
- Added short lifespan wallet heuristics.
- Added low transaction diversity and low transaction entropy penalties.
- Added high contract interaction ratio penalty.
- Added positive scoring signals for Alchemy enrichment, NFT activity, healthy transaction diversity, and high transaction entropy.
- Updated risk flags for bot-like activity:
 - ``low_transaction_diversity``
 - ``low_transaction_entropy``
 - ``high_contract_interaction_ratio``
 - ``hourly_burst_activity``
 - ``daily_burst_activity``
 - ``repeated_contract_loops``
 - ``short_lifespan_wallet``

Documentation + tests

- Updated `docs/scoring-rules.md` with the new Week 5 scoring rules.
- Added `docs/week5-advanced-bot-detection.md` summarizing data sources, features, heuristics, and deliverables.
- Updated `.env.example` for Alchemy configuration.
- Added tests for advanced feature extraction and Week 5 bot detection scoring behavior.
- Confirmed the backend test suite passed with 38 tests.